

How do you run CS223 labs on a Mac without Vivado?

19 September 2026 · M2, 8 GB, macOS 15 [PDF](#) [the five project files](#) section 1 prerequisites, 2 the one-line install, 3 the setup by hand, 4 to 8 your lab and the limits

Apple Silicon Mac (M1 or later), a Digilent Basys3, free tools. Everything here was run on an M2 with 8 GB of RAM and macOS 15 on 19 September 2026. Intel Macs are untested and the installer refuses them.

Two ways in. **Take the one line.** Build by hand only if you want to see each tool go in, or the installer refuses your machine.

	one line	by hand
do	section 1, then 2, then 5 to 8	section 1, then 3, then 4 to 8
time after section 1	one command; 3 min 37 s on the tested machine and fast wifi, 1.4 GB downloaded	5 steps; about 5 minutes of compiling plus 1.76 GB of full clones, call it 20 minutes
you get	the dewfpga command	the same tools plus a Makefile you own
tool versions	two pinned commits, Homebrew versions in section 9	the same two commits, typed by you

Section 4 is the Makefile project the by-hand path uses. Sections 5 to 8 are for both: your own lab, VS Code, the one course file that breaks, what this chain cannot do.

If a step fails. Read the last lines in the terminal. Every error hit during this setup is filed under its exact text, with the fix: nosey-dewdrop.github.io/dewfpga/errors. Not there? Open an issue at github.com/nosey-dewdrop/dewfpga/issues with the last ten lines and the section number.

No Mac or no board with you? The [browser simulator](#) runs the same three files (design.sv, tb.sv, the .xdc) with nothing installed: a virtual Basys3, and a testbench view that draws the waveform.

Contents

- [1. What do you need before you start?](#)
- [2. How do you install it in one line?](#)
- [3. How do you build the chain by hand?](#)
- [4. What is in the project folder?](#)
- [5. How do you use it for your own lab?](#)
- [6. How do you set up VS Code?](#)
- [7. Which course file breaks?](#)
- [8. What can it not do?](#)

- 9. Which versions were tested?
-

1. What do you need before you start?

The board. A Digilent Basys3 and its micro-USB cable. Apple Silicon MacBooks have only USB-C ports, so you also need a USB-C to USB-A adapter or hub. The board is powered from the cable; leave the JP2 jumper on USB.

The terminal. Press Cmd Space, type Terminal, press Enter. Every command in this guide goes into that window: paste it, press Enter, wait until the line ending in % comes back before you paste the next one. Some commands span several lines and end in \; copy the whole grey block at once.

Apple's command line tools. They give your Mac git, make and the C compiler.

```
xcode-select --install
```

A window pops up; click Install and wait. If instead the terminal prints xcode-select: note: Command line tools are already installed, that is fine. Check:

```
xcode-select -p
# /Library/Developer/CommandLineTools
```

Homebrew. It installs developer tools by name.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It asks for your Mac password; the letters do not show while you type. At the end it prints a few lines under "Next steps". Paste those into the terminal too, or the next command answers command not found. Check:

```
brew --version
# Homebrew 4.x
```

VS Code. Download it from code.visualstudio.com. Then in VS Code press Cmd Shift P, type shell command, choose "Install 'code' command in PATH". After that code . in the terminal opens the current folder. Do not edit these files in TextEdit; it saves rich text, and the tools cannot read that.

2. How do you install it in one line?

```
curl -fsSL https://nosey-dewdrop.github.io/dewfpga/install | bash
```

3 min 37 s on the tested machine with fast wifi, 1.4 GB downloaded; on slow wifi it takes longer, and it prints progress the whole time. It never asks for a password. It puts the dewfpfga command into ~/.dewfpfga (its sha256 is compared with nosey-dewdrop.github.io/dewfpfga/dewfpfga.tgz.sha256, so a broken download stops here), links it into Homebrew's bin, installs Yosys, openFPGALoader, Icarus Verilog and the build tools of section 3.1 through Homebrew, and builds nextpnr-xilinx and prjxray into ~/fpga at the two commits in section 9. Nothing touches your system Python. The last lines it prints:

```
all good.

Total: 217 s. Log: /Users/you/fpga/install.log
Next:  dewfpfga new blink && cd blink && dewfpfga flash
```

To read the script before running it:

```
curl -fsSL https://nosey-dewdrop.github.io/dewfpfga/install -o install.sh
code install.sh      # read it, then:
bash install.sh
```

Check that every piece is there:

```
dewfpfga check
```

```
dewfpfga check  (FPGA_HOME=/Users/you/fpga)
✓ yosys          /opt/homebrew/bin/yosys
✓ iverilog       /opt/homebrew/bin/iverilog
✓ openFPGALoader /opt/homebrew/bin/openFPGALoader
✓ nextpnr-xilinx /Users/you/fpga/nextpnr-xilinx/build/nextpnr-xilinx
✓ xc7frames2bit  /Users/you/fpga/prjxray/build/tools/xc7frames2bit
✓ fasm2frames    /Users/you/fpga/prjxray/utils/fasm2frames.py
✓ chipdb         /Users/you/fpga/chipdb/xc7a35t.bin
✓ venv           /Users/you/fpga/venv/bin/python
✓ python pkgs    fasm prjxray pyyaml textx simplejson intervaltree
all good.
```

A red x names the missing piece: run the curl line again (it is the same as dewfpfga install), it skips what is done. dewfpfga: command not found has two causes: the Homebrew "Next steps" lines from section 1 were not pasted (paste them, open a new terminal window), or the installer stopped before its last line; scroll up to the first ERROR: line and look it up on the errors page.

Now plug the Basys3 into the Mac with the USB cable (the PROG port, next to the power switch) and flip the power switch on. Make the example project and flash it:

```
dewfpga new blink      # prints: blink/ ready: cd blink && dewfpga flash      (VS Code: %
1B)
cd blink
dewfpga sim            # simulation
dewfpga bit            # .sv -> .bit
dewfpga flash          # .bit -> the board; runs bit first when the .bit is missing or o
lder than the .sv
```

What each one prints when it works. `dewfpga sim`:

```
VCD info: dumpfile blink.vcd opened for output.
PASS: 3 checks (see the waveform for the toggle)
blink_tb.sv:35: $finish called at 200000 (1ps)
```

`dewfpga bit`, three lines: the pin check, place and route, the bitstream. The full place-and-route log is in `blink.log`.

```
xdc ok: 33 ports, all mapped.
pnr ok: 73 LUT, 27 FF, 278.71 MHz (PASS at 100.00 MHz)    (full log: blink.log)
blink.bit 2.2 MB
```

`dewfpga flash`:

```
Load SRAM: [=====] 100.00%
Done
ir: 1 isc_done 1 isc_ena 0 init 1 done 1
```

done 1 means the chip accepted the design. Flip switch 0 up (SW0, the rightmost of the 16 switches): LED 15 (LD15, leftmost) lights and LED 0 (LD0, rightmost, just above SW0) blinks once a second. The design lives in SRAM, so a power cycle clears it; flash again.

The rules `dewfpga` follows in a folder:

- Every `.sv` and `.v` in the folder is synthesized, so submodules can live in their own files. Files ending in `_tb.sv` are testbenches and stay out of the bitstream. One design per folder.
- The top module is the `.sv` whose name matches the `.xdc`; if there is only one `.sv`, that one. Otherwise name it: `dewfpga bit lab4`.
- The pin file is `<top>.xdc`, or the only `.xdc` in the folder. Pins in it that the design does not use are ignored, so a fully uncommented `Basys3_Master.xdc` is fine.
- `dewfpga sim` needs `<top>_tb.sv`. `bit` and `flash` do not. `flash` builds the `.bit` first if it is missing or older than the sources, so `dewfpga flash` alone is enough.
- The top module must have its file's name: `lab4.sv` holds module `lab4`. Submodule files can be named anything.
- `dewfpga bit` writes no bitstream when timing is not met. `dewfpga sim` exits with an error when the testbench prints `$error` or `$fatal`, so a red run is visible.

dewfpga install	install the toolchain (safe to re-run)
dewfpga check	is every piece in place
dewfpga sim bit flash clean [top]	work on the .sv files in the current folder
dewfpga new <dir>	blink example with a VS Code task (Cmd Shift B = f
lash)	
dewfpga uninstall	remove the tools in ~/fpga, ~/.dewfpga and the lin
k; brew packages stay	

Continue with [section 5](#).

3. How do you build the chain by hand?

These are the commands the installer runs, at the same pinned commits. Vivado does six jobs for a lab; one free tool does each one here:

```

your_design.sv
|-> Icarus Verilog    -> simulation
|-> Yosys             -> synthesis
|-> nextpnr-xilinx    -> place and route
|-> fasm2frames        -> configuration frames
|-> xc7frames2bit      -> your_design.bit
|-> openFPGALoader    -> the board, over USB

```

Three come from Homebrew, two you compile, and fasm2frames is a Python script inside the prjxray checkout. nextpnr also needs a chip database, generated once by another Python script. Everything lands in ~/fpga: 1.4 GB with the installer's shallow clones, 1.76 GB with the full clones below. The Makefile in section 4 expects exactly this layout: ~/fpga/venv, ~/fpga/nextpnr-xilinx/build, ~/fpga/chipdb/xc7a35t.bin, ~/fpga/prjxray.

3.1 Homebrew packages

```
brew install yosys openfpgaloader icarus-verilog cmake ninja eigen pkg-config python@3.14
```

The first three are tools from the diagram; the rest are what the two builds below need. A few hundred MB. Check:

```

yosys -V                # Yosys 0.69+post
openFPGALoader --Version # v1.1.1
iverilog -V | head -1    # Icarus Verilog version 13.0 (stable)

```

Newer numbers are expected; only the ones in section 9 were tested.

3.2 Python venv

fasm2frames (3.5) and the chip database generator (3.4) are Python scripts. Homebrew's Python refuses a plain `pip install` ([externally-managed-environment](#)), so they get a private one.

```
mkdir -p ~/fpga
"${brew --prefix}/opt/python@3.14/bin/python3.14" -m venv ~/fpga/venv
~/fpga/venv/bin/pip install fasm==0.0.2.post88 pyyaml==6.0.3 textx==4.4.0 simplejson==
4.1.2 intervaltree==3.2.1 numpy==2.5.3 pyjson5==2.0.1
```

Check (prints nothing when it works):

```
~/fpga/venv/bin/python -c "import fasm, yaml, textx, simplejson, intervaltree"
```

3.3 nextpnr-xilinx

Place and route. Homebrew has no macOS build, so you compile it. The clone is most of the download; the compile is about 1.5 minutes on this machine.

```
cd ~/fpga
git clone --recursive https://github.com/openXC7/nextpnr-xilinx.git
cd nextpnr-xilinx
git checkout 3fd78784c7788f93f276358edf5477221cc6c179
git submodule update --init --recursive
cmake -B build -G Ninja \
  -DARCH=xilinx -DCMAKE_BUILD_TYPE=Release \
  -DUSE_OPENMP=OFF -DBUILD_GUI=OFF -DBUILD_PYTHON=OFF -DBUILD_TESTS=OFF
ninja -C build -j3 nextpnr-xilinx bbasm
```

`-DUSE_OPENMP=OFF` is required: Apple's compiler has no OpenMP ([fopenmp](#)). `-j3` fits 8 GB of RAM; with 16 GB or more use `-j$(sysctl -n hw.ncpu)`. Check:

```
~/fpga/nextpnr-xilinx/build/nextpnr-xilinx --version
# "nextpnr-xilinx" -- Next Generation Place and Route (Version 0.9.6)
```

3.4 Chip database

nextpnr needs a description of the XC7A35T. It is not in the download; you generate it once ([no-chipdb](#)). About 40 seconds and 900 MB of RAM while it runs.

```
mkdir -p ~/fpga/chipdb
cd ~/fpga/nextpnr-xilinx
~/fpga/venv/bin/python xilinx/python/bbaexport.py \
  --xray xilinx/external/prjxray-db/artix7 \
  --metadata xilinx/external/nextpnr-xilinx-meta/artix7 \
  --device xc7a35tcp236-1 \
  --constids xilinx/constids.inc \
  --bba ~/fpga/chipdb/xc7a35t.bba
build/bbasm -l ~/fpga/chipdb/xc7a35t.bba ~/fpga/chipdb/xc7a35t.bin
rm ~/fpga/chipdb/xc7a35t.bba
```

Check:

```
ls -lh ~/fpga/chipdb/xc7a35t.bin
# about 90 MB
```

Another 7-series board would need its own `--device`. Untested.

3.5 prjxray

Turns nextpnr's text output into the binary `.bit`: one Python script, one compiled tool. About 2 minutes.

```
cd ~/fpga
git clone --recursive https://github.com/f4pga/prjxray.git
cd prjxray
git checkout c9f02d8576042325425824647ab5555b1bc77833
git submodule update --init --recursive
~/fpga/venv/bin/pip install --no-deps -e .
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -DPRJXRAY_BUILD_TESTING=OFF
cmake --build build --target xc7frames2bit -j3
```

`--recursive` is required ([yaml-cpp-not-built](#)). `-DCMAKE_POLICY_VERSION_MINIMUM=3.5` is required ([cmake-policy-version-minimum](#)). prjxray's own README says to install Vivado 2017.2; that is for mapping a new chip, and ours is mapped. Ignore it. Check:

```
ls ~/fpga/prjxray/build/tools/xc7frames2bit ~/fpga/prjxray/utils/fasm2frames.py
```

Both paths print back. The full check is `make check` in the next section, one line per tool.

4. What is in the project folder?

A project is one folder. Download the files, do not type them:

```
mkdir -p ~/cs223/blink && cd ~/cs223/blink
curl -L -o blink.zip https://nosey-dewdrop.github.io/dewfpga/blink.zip && unzip -q blink.zip && rm blink.zip && ls -A
# .vscode Makefile blink.sv blink.xdc blink_tb.sv check_xdc.py
```

file	what it is
blink.sv	the design: LED 0 blinks once a second while switch 0 is on, LED 15 follows switch 0
blink_tb.sv	the testbench: three checks, prints PASS or FAIL

file	what it is
blink.xdc	Digilent's Basys3 master XDC with the 33 pin lines for clk, sw and led uncommented. Vivado reads the same file
check_xdc.py	compares the design's ports with the XDC before place and route, because nextpnr's own error names the wrong port
Makefile	runs the six tools in order. <code>--seed 1</code> makes every build byte-identical
<code>.vscode/tasks.json</code>	<code>Cmd Shift B</code> in VS Code runs <code>make flash</code>

dewfpga new blink gives the same `blink.sv`, `blink_tb.sv` and `blink.xdc` without the `Makefile`, and its `tasks.json` says `dewfpga flash` instead.

If you copied the `Makefile` from somewhere else and `make` says `missing separator`, the tabs became spaces: [missing-separator](#).

Plug the board in over USB, switch it on, then from inside the folder:

```
make check
```

```
yosys          /opt/homebrew/bin/yosys
iverilog       /opt/homebrew/bin/iverilog
openFPGALoader /opt/homebrew/bin/openFPGALoader
nextpnr-xilinx /Users/you/fpga/nextpnr-xilinx/build/nextpnr-xilinx
xc7frames2bit  /Users/you/fpga/prjxray/build/tools/xc7frames2bit
chipdb         /Users/you/fpga/chipdb/xc7a35t.bin
fasm2frames    /Users/you/fpga/prjxray/utils/fasm2frames.py
venv           /Users/you/fpga/venv/bin/python
```

Eight lines, each ending in a path, and no `-MISSING-` anywhere: done. A line such as `chipdb -MISSING-` names the step of section 3 to redo (3.4 here). Then:

```
make sim      # PASS: 3 checks
make bit      # xdc ok / pnr ok / blink.bit, about 4 seconds
make flash    # Load SRAM ... done 1
```

The outputs are the same three blocks shown in section 2. `make sim` also writes `blink.vcd`. No waveform viewer is installed by this guide; to look at the waveform, open the [browser testbench](#) and paste the same `blink.sv` and `blink_tb.sv` there, it draws it.

5. How do you use it for your own lab?

The pin file. The course hands out `Basys3_Master.xdc` with every line commented out (a copy: [templates/Basys3_Master.xdc](#)). Copy it into your lab folder and delete the leading `#` on the `set_property` lines for the pins your module uses. Uncommenting all of them is fine too: pins the

design does not use are ignored. The port names in your top module must match the names in that file: `clk`, `sw[0]`, `led[0]`, `seg[0]`, `an[0]`, `btnC`. If your module says `clock`, change the name inside `[get_ports ...]` on that line. The xdc check that runs before place and route lists every port that has no pin.

With dewfpga: put the `.sv` files and the `.xdc` in one folder, `cd` into it, `dewfpga flash`. With `lab4.sv` and `Basys3_Master.xdc` in the folder that is all. With several `.sv` files, name the top: `dewfpga flash lab4`.

With the Makefile: copy `Makefile` and `check_xdc.py` from section 4 into the lab folder and change the first three lines.

```
TOP := traffic_light
SRCS := traffic_light.sv debounce.sv
XDC := Basys3_Master.xdc
```

`TOP` is the top module's name, `SRCS` every `.sv` in the design, `XDC` the pin file.

Either way, simulation looks for `<top>_tb.sv`; the bitstream does not need one. If bit stops on the course's `SevSeg_4digit.sv`, section 7 has the one-line fix.

6. How do you set up VS Code?

Syntax colours and error marks for SystemVerilog:

```
code --install-extension mshr-h.veriloghdl
```

`dewfpga new` and `blink.zip` both put a `.vscode/tasks.json` in the project, so `Cmd Shift B` builds and flashes the open design. For a folder you made yourself, copy [templates/.vscode/tasks.json](#) into it; the file says `make flash`, change that to `dewfpga flash` if you took the one-line path.

7. Which course file breaks?

One. `SevSeg_4digit.sv` from the course has a port line that Vivado and Icarus accept and Yosys does not (port-neither-input-nor-output).

```
output [6:0]seg, logic dp,      // dp inherits `output` from the previous port, per the
standard
```

Yosys does not implement that inheritance. Write it as two lines and it synthesizes.

```
output [6:0] seg,
output logic dp,
```

What else was tried: four modules from old student repos, together in one design. An FSM with `typedef enum`, a 16x8 RAM, a debouncer and the seven-segment driver above: 136 LUTs, 48 flip-flops, timing passes at 154.70 MHz against the 100 MHz clock, bitstream in 4.5 seconds. Language features in those files that both Icarus (`sim`) and Yosys (`bit`) accepted: `parameter`, `generate`, `struct packed`, `$clog2`, `unique case`, packed arrays, interface with `modport`. A submodule file named differently from its module, Turkish characters and a BOM in the source, and spaces in the folder path all went through too. That is four files, not the whole course.

8. What can it not do?

This covers the part of Vivado that CS223 uses, and no more.

- **Nothing from Vivado's IP Catalog.** No Block Design, MicroBlaze, AXI, Clocking Wizard, or the BRAM, VGA and UART cores. 35 old student repos with 240 source files were searched for those; none use any of it. (Searched, not synthesized.)
 - **No GUI.** No waveform viewer, schematic or in-chip debugger. `sim` writes a `.vcd`; the browser testbench draws the waveform for the same files.
 - **Two parsers.** Icarus reads your code for simulation, Yosys for synthesis, and they do not support the same SystemVerilog. Code that passes `sim` can fail in `bit`; section 7 is the one case hit so far, and rewriting the line fixed it.
 - **The timing number is nextpnr's estimate.** Vivado's report is the official one. In the two designs above the margin was 1.5x (four modules) and 2.8x (blink) over 100 MHz.
 - **Unofficial tools.** AMD does not make or support them; prjxray worked out the bitstream format without AMD's documentation. If a lab is graded in Vivado, open it in Vivado once before the demo.
 - **Memories become distributed RAM** (`-nobram`). The 16x8 RAM in section 7 mapped to four RAM32M cells. A large memory such as a VGA framebuffer would not fit this way; big multipliers and DSP blocks are untested.
 - **One board, one platform.** Basys3 (XC7A35T), Apple Silicon. Intel Mac, Linux and other boards are untested and the installer refuses them.
 - **Confirmed on the board:** `blink`, switches-to-LEDs, and the Lab 2 adder/subtractor. The four-module design in section 7 went as far as a bitstream and was not loaded.
-

9. Which versions were tested?

The installer pins `nextpnr-xilinx` and `prjxray` to these commits; section 3 checks out the same ones. Homebrew packages cannot be pinned; the versions below are what was tested.

Yosys	0.69+post (git 143eb14f)
nextpnr-xilinx	0.9.6, openXC7 commit 3fd7878
prjxray	commit c9f02d8; prjxray-db 0.9.1-30-g1768fb3 (artix7)
openFPGALoader	1.1.1
Icarus Verilog	13.0 (stable)
Python	3.14 (Homebrew), venv under ~/fpga
Board	Digilent Basys3, Artix-7 XC7A35T-1CPG236C
Machine	Apple M2, 8 GB RAM, macOS 15 (Darwin 24.2.0)
